

STM32 Flight Computer

An autonomous quadrotor autopilot in `no_std` Rust

A complete technical dossier of the firmware, state estimation, control, navigation, actuation, telemetry, safety architecture, and ground/companion software — written to be read once and understood deeply.

TARGET MCU

STM32H723ZGT6 · Cortex-M7F @ 400 MHz

AIRFRAME

Quad-X · 4× DShot ESC · 4× PWM servo payload

CONTROL

Cascaded attitude→rate PID @ 500 Hz

MISSION

Waypoints, RTH, position hold, autonomous weed extraction

LANGUAGE / RUNTIME

Rust · Embassy async, `no_std`

ESTIMATION

Madgwick AHRS + complementary NED filter

COMMS

Hand-rolled MAVLink v2 · Raspberry Pi relay · Python GCS

CODEBASE

~7k lines firmware + ~4k lines ground/Pi

Prepared as interview-preparation study material · Reflects the firmware as committed on branch **main**.

Contents

1. How to read this document

- 1.1 What the machine is, in one paragraph
- 1.2 The honest status of the project

2. System architecture & data flow

- 2.1 The three-computer topology
- 2.2 End-to-end signal path
- 2.3 The shared-state pattern

3. The hardware platform

- 3.1 MCU, memory, and why the H7
- 3.2 Sensors, actuators, and buses
- 3.3 Pin map & peripheral reservations

4. Software foundation: Rust + Embassy

- 4.1 Why `no_std` Rust for flight code
- 4.2 The async cooperative executor
- 4.3 Task table & timing budget
- 4.4 Boot sequence and clock tree

5. Sensor drivers

- 5.1 IMU — ICM-42688-P
- 5.2 Barometer — MS5611
- 5.3 Magnetometer — QMC5883L
- 5.4 GPS — u-blox M10 / UBX
- 5.5 Optical flow & rangefinder — MTF-02P
- 5.6 RC — SBUS
- 5.7 Battery & current — ADC
- 5.8 Freshness: the `usable()` discipline

6. State estimation

- 6.1 Reference frames & the NED/NWU trap
- 6.2 Madgwick AHRS
- 6.3 Complementary position/velocity estimator

7. Flight control

- 7.1 The cascade: angle → rate → torque
- 7.2 PID implementation details
- 7.3 Quad-X mixing & desaturation

8. Navigation & flight modes

- 8.1 Mode set and demotion ladder
- 8.2 Guidance law
- 8.3 The weed-extraction state machine
- 8.4 Geofence, failsafes, and latching

9. Actuation: the DShot pipeline

- 9.1 DShot on the wire
- 9.2 Timer burst-DMA encoding
- 9.3 The H7 DMA gotchas
- 9.4 Bidirectional DShot / eRPM

10. Telemetry: MAVLink v2 by hand

- 10.1 Frame format & CRC

10.2 The staggered TX schedule

10.3 The RX state machine & command dispatch

10.4 Mission upload protocol

11. Safety architecture (read this twice)

11.1 The layered kill-path model

11.2 Arming gates

11.3 In-flight failsafes

11.4 The hardware watchdog

12. Companion computer & ground station

12.1 The Pi relay

12.2 The Python GCS & computer-vision pipeline

13. Build system & feature flags

14. Known issues & engineering judgement

15. Interview preparation: likely questions & strong answers

16. Glossary

1 How to read this document

This is written for someone who will walk into an aerospace-software interview and be asked to explain a real, embedded, safety-relevant flight system that they built. It assumes you know how to program and understand computers at a systems level, but it does *not* assume you already remember every decision baked into this firmware. Everything you need to reason about the machine — from the clock tree to the reference-frame conventions to the exact reason a DMA transfer silently fails on an STM32H7 — is here.

Each major subsystem is presented three ways: **what it does** (the behaviour), **how it works** (the mechanism, with the actual code), and **why it is built that way** (the engineering trade-off, which is what interviewers actually probe). Coloured callouts flag the three things worth memorising:

INTERVIEW ANGLE

A point that makes a strong talking point or answers a common question.

GOTCHA

A subtle bug, hardware trap, or convention that bites people — being able to explain these signals depth.

SAFETY-CRITICAL

A behaviour where getting it wrong hurts hardware or people.

1.1 • What the machine is, in one paragraph

It is an autonomous quadrotor autopilot. A single STM32H723 microcontroller running Rust reads a 6-axis IMU at 500 Hz, fuses it with a magnetometer, barometer, GPS, and optical-flow sensor into an attitude and position estimate, runs a cascaded PID controller to hold a commanded attitude, mixes that into four motor commands sent over the DShot digital ESC protocol, and layers autonomous behaviour on top: hold position, fly GPS waypoint missions, return to home, land, follow a person, and perform a novel autonomous "weed extraction" mission where it flies to a GPS coordinate, descends, closes a gripper, and drops the payload in a bin. It talks to the operator over MAVLink through a Raspberry Pi relay to a Python ground station that also runs the computer-vision that finds the weeds and people. The whole thing is defensively engineered around the assumption that any sensor, link, or task can die at any moment.

1.2 • The honest status of the project

Being candid about state is itself an interview asset. The **firmware is feature-complete**; the remaining work is hardware bring-up. Motors spin under DShot300; the IMU, GPS, and estimator are bench-verified; the frame conventions have been physically checked. Some things are explicitly in a bring-up posture and you should know them:

- The remove-before-flight arming gate is *currently disabled in software* (`SAFETY_PIN_ENABLED = false`) for bench work — it must be re-enabled before field flights.
- The build is often compiled with `rc-optional` (radio was lost), `range-alt` (rangefinder substituting for a barometer), or `no-batt` feature flags, each of which trades a safety layer for the ability to bench-test — the firmware is explicit and loud about what each removes.
- There is one genuinely open bug — an occasional executor scheduling "cadence clumping" — which is instrumented but not yet root-caused. Section 14 discusses it; being able to describe an *unsolved* problem and your debugging strategy is often more impressive than a tidy success.

2 System architecture & data flow

2.1 · The three-computer topology

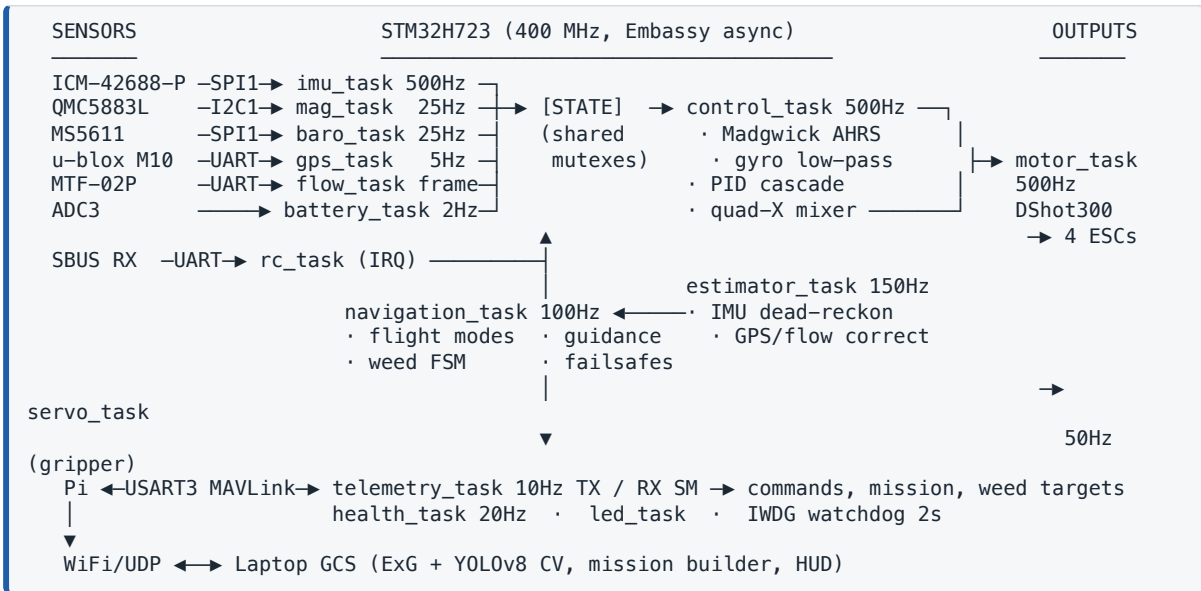
The system is deliberately split across three processors, each doing what it is good at:

Node	Hardware	Responsibility	Why here
Flight computer	STM32H723, Cortex-M7F @ 400 MHz	Hard-real-time: sensor sampling, fusion, control, actuation, safety.	Deterministic timing, direct peripheral access, no OS jitter.
Companion	Raspberry Pi (Zero 2 W flight / Pi 4 bench)	Soft-real-time relay: bridges the FC's serial MAVLink to WiFi/UDP, streams H.264 camera, forwards weed/person targets, sends a heartbeat.	Linux networking + camera + a spare CPU, but not trusted with control.
Ground station	Laptop, Python + DearPyGUI	Operator UI, telemetry HUD, mission builder, and the computer vision (ExG weed detection + YOLOv8 person tracking).	Heavy compute (PyTorch) and a screen; latency-tolerant.

INTERVIEW ANGLE

The key architectural principle: **control authority lives only on the deterministic node**. The Pi and laptop can *request* things (arm, change mode, supply a target coordinate) but the STM32 independently validates every request and owns every motor command. If the Pi or laptop vanishes, the aircraft is still airworthy and, in fact, treats the disappearance of the Pi as a failsafe trigger.

2.2 · End-to-end signal path



Every arrow that crosses into the STM32 box is a task; tasks never call each other, they only read and write the shared `STATE`.

2.3 · The shared-state pattern

There is no message passing between tasks. Instead there is one global `STATE` struct — a collection of `embassy_sync::Mutex`-wrapped fields — and every task is a producer of some fields and a consumer of others. This is the single most important structural idea in the firmware.

```
pub struct SharedState {
    pub attitude:      Mutex<CriticalSectionRawMutex, Quaternion>,
```

```

pub imu_data:      Mutex<CriticalSectionRawMutex, ImuData>,
pub baro_data:     Mutex<CriticalSectionRawMutex, BaroData>,
pub gps_fix:       Mutex<CriticalSectionRawMutex, GpsFix>,
pub rc_input:      Mutex<CriticalSectionRawMutex, RcInput>,
pub nav_command:   Mutex<CriticalSectionRawMutex, NavCommand>,
pub motor_outputs: Mutex<CriticalSectionRawMutex, MotorOutputs>,
pub pos_estimate:  Mutex<CriticalSectionRawMutex, PosEstimate>,
pub sensor_health: Mutex<CriticalSectionRawMutex, SensorHealth>,
pub armed:         Mutex<CriticalSectionRawMutex, bool>,
// ...weed_target, mode_override, home_override, motor_test, servo_outputs...
}

```

Data flows one direction through it. For example `imu_task` is the sole writer of `imu_data`; `control_task`, `estimator_task`, and `health_task` are readers. Because the executor is single-core and cooperative (Section 4), a `.lock().await` can only yield at an explicit await point, so critical sections are short and deadlock between the two-field accesses is avoided by never holding two locks across an await unnecessarily.

GOTCHA

The `CriticalSectionRawMutex` matters: it disables interrupts for the (very short) duration the lock is held, which is what makes the shared state safe to touch from both the thread-mode executor *and* the panic handler / any interrupt-mode code. A plain non-interrupt-safe mutex would be a latent race.

3 The hardware platform

3.1 • MCU, memory, and why the H7

Part	STM32H723ZGT6 — single-core Arm Cortex-M7F with double-precision FPU
Clock	400 MHz SYSCLK, 200 MHz AHB, 100 MHz APB (see §4.4 for the PLL derivation)
Flash	1 MB @ 0x0800_0000
RAM used	AXI-SRAM, 320 KB @ 0x2400_0000 — chosen because it is DMA-reachable
Notably <i>not</i> used	DTCM @ 0x2000_0000 — fast, but invisible to the DMA controllers on the H7

GOTCHA — THE CLASSIC H7 DMA TRAP

On the STM32H7, the tightly-coupled DTCM RAM cannot be reached by the general DMA engines. If the linker places a DMA buffer there, the transfer silently moves nothing — no fault, no error flag, just dead peripherals. This firmware puts all statics in AXI-SRAM and the DShot debug build actively asserts the motor buffer's address is 0x24xx_xxxx. Being able to say "on H7 you must check your DMA buffer isn't in DTCM" is a strong embedded-systems signal.

The M7F is chosen for headroom: a 500 Hz control loop with a Madgwick filter (several dozen float multiply-adds), a 150 Hz estimator with a full quaternion rotation matrix, and a 100 Hz navigation task with trig and haversine math all run comfortably with the hardware FPU, leaving the CPU mostly idle.

3.2 • Sensors, actuators, and buses

Function	Part	Bus / rate	Notes
IMU (6-axis)	ICM-42688-P	SPI1 @ 12.5 MHz, 500 Hz	±2000 dps, ±16 g, 1 kHz ODR. WHO_AM_I = 0x47.
Barometer	MS5611	SPI1 (shared), 25 Hz	Altitude via ISA model, relative-to-launch datum.
Magnetometer	QMC5883L	I2C1 @ 400 kHz, 25 Hz	Tilt-compensated heading; boot-time figure-8 cal.
GPS	u-blox M10 (SEQUIRE)	USART1 @ 115200, 5 Hz	UBX NAV-PVT binary, configured over the same UART.
Optical flow + range	MTF-02P	UART4 @ 115200, ~50 Hz	MICOLINK protocol; body-frame velocity + downward range.
RC receiver	ELRS → SBUS	USART2 @ 100 kbaud 8E2, inverted	16 channels; hardware-inverted RX in firmware.
ESCs ×4	4-in-1 BLHeli_S	TIM3 CH1–4, DShot300	Digital ESC protocol via timer burst-DMA.
Servos ×4	Gripper payload	TIM4 CH1–4, 50 Hz PWM	1000–2000 μs standard servo pulse.
Battery / current	Resistive divider / ESC CUR pad	ADC3 (PC0 / PF3), 2 Hz	Per-cell voltage → SoC; critical-voltage failsafe.
Telemetry	Raspberry Pi	USART3 @ 57600, full-duplex	MAVLink v2 both directions + Pi heartbeat watchdog.

INTERVIEW ANGLE

Note the deliberate bus diversity. Two devices share **SPI1** (fast, deterministic, good for the 500 Hz IMU) behind a mutex; the slow compass sits on **I2C**; every serial sensor gets its own **UART with ring-buffered DMA RX** so bytes are never dropped while the CPU is busy elsewhere. The motors get a **timer + DMA** path so the CPU spends no cycles bit-banging a 300 kbit/s protocol.

3.3 · Pin map & peripheral reservations

DMA channels are a finite resource and are allocated up front so future additions don't collide:

Peripheral	Use	DMA
SPI1	IMU + baro	DMA2 CH3 (TX), CH0 (RX)
USART1	GPS	DMA2 CH6/CH5
USART2	SBUS RC	DMA1 CH5 (RX)
USART3	Telemetry	DMA1 CH3/CH1
UART4	Optical flow	DMA1 CH2 (RX)
TIM3	Motors (DShot)	DMA1 stream 4, DMAMUX req 27 (TIM3_UP)
TIM4 / ADC3 / I2C1	Servos / battery / mag	none (PWM / blocking)
IWDG1	Independent watchdog	2.0 s timeout, petted by control_task

4 Software foundation: Rust + Embassy

4.1 · Why `no_std` Rust for flight code

The firmware is `#![no_std]` and `#![no_main]`: no operating system, no heap allocator, no standard library — just the core language, the hardware, and a handful of embedded crates. The argument for Rust here is exactly the argument aerospace makes for MISRA-C and static analysis, but enforced by the compiler:

- **Memory safety without a garbage collector.** No buffer overruns, no use-after-free, no dangling pointers — the borrow checker rejects them at compile time, and there is no GC to introduce unbounded pauses.
- **Data-race freedom.** Sharing state between tasks requires a `Sync` type; the compiler will not let you share a bare mutable global. The `STATE` mutex pattern isn't convention, it's the only thing that compiles.
- **Explicit fallibility.** Sensor reads return `Result / Option`; you cannot accidentally use a failed SPI transfer's result — the type system forces you to handle the `None`.
- **`unsafe` is a searchable keyword.** The few places that touch raw registers (the DShot DMA setup, the watchdog, the panic handler) are marked `unsafe` and are auditable in isolation.

INTERVIEW ANGLE

A crisp way to put it: "Rust gives me the deterministic, allocation-free execution model that flight code needs, while moving an entire class of memory and concurrency bugs from runtime — where in a drone they mean a crash — to compile time." Then acknowledge the cost: a steeper learning curve, a less mature embedded ecosystem than C, and needing `unsafe` plus register-level knowledge for the peripherals the HAL doesn't cover.

4.2 · The async cooperative executor

Embassy is an `async / await` runtime for microcontrollers. Instead of an RTOS with preemptive threads and stacks, each "task" is an `async fn` compiled into a state machine. A single-core cooperative *executor* polls ready tasks; a task runs until it hits an `.await` that isn't ready (a timer tick, a DMA completion, a UART byte), at which point it yields and the executor runs the next ready task. When nothing is ready, the core sleeps (WFE) until an interrupt wakes it.

The consequences that matter for this design:

- **No per-task stacks.** An `async` state machine's locals live in a compact generated struct, not a reserved stack — dramatically less RAM than an RTOS thread-per-task.
- **Cheap concurrency.** A task that is "waiting 40 ms for the next baro conversion" or "waiting for 25 SBUS bytes" costs nothing while it waits; the CPU is free.
- **Cooperative, not preemptive.** A task that never awaits would starve everything else. This is a discipline the code must honour — and is the suspected culprit of the open cadence bug (§14).
- **Timing via `Ticker`.** A periodic task holds a `Ticker::every(Duration::from_hz(500))` and awaits `ticker.next()`; the `Ticker` tracks absolute deadlines so periods don't drift with execution time.

4.3 · Task table & timing budget

Task	Rate	Role	Writes
<code>control_task</code>	500 Hz	AHRS fusion → PID cascade → mixer; pets watchdog; crash cutoff	attitude, motor_outputs
<code>imu_task</code>	500 Hz	SPI burst-read, scale, de-bias gyro	<code>imu_data</code>

Task	Rate	Role	Writes
motor_task	500 Hz	Encode + DMA DShot frames; ESC arming	ESC pins
estimator_task	150 Hz	IMU dead-reckoning + GPS/flow correction	pos_estimate
navigation_task	100 Hz	Flight-mode logic, guidance, weed FSM, failsafes	nav_command, servo, weed
arming_task	50 Hz	Arm/disarm, RBF pin, RC watchdog	armed, flight state
servo_task	50 Hz	TIM4 PWM duty from servo_outputs	servo pins
baro_task	25 Hz	D1/D2 conversions, compensation, ISA altitude	baro_data
mag_task	25 Hz	Read, calibrate, tilt-compensate heading	mag_data
health_task	20 Hz	Per-sensor plausibility → arming gates	sensor_health
telemetry_task	10 Hz TX	MAVLink TX schedule + RX state machine (two joined futures)	many (via RX)
gps_task	5 Hz	UBX NAV-PVT parse	gps_fix
battery_task	2 Hz	ADC → voltage/SoC, critical hysteresis	battery
flow_task	frame-driven	MICOLINK parse	flow
rc_task	event	SBUS frame parse w/ idle detection	rc_input, mode_override
led_task	1–5 Hz	Status blink pattern per state	LED

INTERVIEW ANGLE — SEPARATION OF ESTIMATION AND CONTROL

Note that `imu_task`, `control_task`, and `estimator_task` are separate. Sampling, the attitude/rate control loop, and the position estimator each run at their own rate and communicate only through `STATE`. This is the classic autopilot decomposition (sense → estimate → control → actuate) and it means, e.g., a stalled GPS can never delay a motor update.

4.4 • Boot sequence and clock tree

`main()` configures the RCC (clock tree) before anything else. The derivation is worth knowing cold because clock-config questions are common:

```

HSI 64 MHz ÷4 = 16 MHz → PLL1 ×50 = 800 MHz VCO
÷P (2) → 400 MHz = SYSCLK
÷Q (8) → 100 MHz = SPI / FDCAN kernel clock
AHB ÷2 → 200 MHz (HCLK)
APB1 ÷2 → 100 MHz (TIM3 timer clock doubles to 200 MHz)
APB2/3/4 ÷2 → 100 MHz

```

Deliberate choices in that block:

- **HSI, not an external crystal.** Using the internal 64 MHz oscillator means the firmware doesn't depend on knowing the board's crystal — it runs on any H723 board. The cost is slightly worse clock accuracy, acceptable here.
- **Power supply config.** `SupplyConfig::Default` and `VoltageScale::Scale1` are set explicitly. This is a hard-won line: the H723's `PWR_CR3` register is *write-once per power-on-reset*, and any supply variant other than the POR default drops the voltage-ready flag and hangs Embassy's init loop forever. This is why the board must be true power-cycled (not soft-reset) after every flash.
- **ADC clock mux.** The ADC kernel clock is explicitly routed from `per_ck`; without it, conversions never complete.

- **Watchdog first.** `init_watchdog()` is called *before* `embassy_stm32::init()`, so that a hang inside HAL init resets the chip instead of bricking it.

GOTCHA — WRITE-ONCE REGISTERS & SOFT RESET

The CR3 write-once behaviour plus a time-driver that hangs on soft reset means "flash, then press reset" leaves the board looking dead. The fix baked into the tooling is: always do a genuine power cycle after flashing. This is exactly the kind of hardware-specific footgun that separates people who've actually shipped embedded firmware from people who've only read about it.

After clocks, `main` initialises the shared SPI1 bus into a global mutex, creates the LED/RBF/grip GPIOs, and then `spawner.spawn(...)`s every task. It ends in an idle `loop { Timer::after(1s) }` — the real work is entirely in the spawned tasks.

5 Sensor drivers

Every driver follows the same shape: configure the device, verify identity, then a `Ticker`-paced loop that reads, converts to SI units, and writes a timestamped snapshot into `STATE`. The timestamp is the linchpin of the freshness discipline (§5.8).

5.1 • IMU — ICM-42688-P

The most important sensor: attitude comes from here. Over SPI1 at 12.5 MHz, the driver soft-resets the chip, confirms `WHO_AM_I == 0x47` (retried three times so one flaky power-up read doesn't permanently fault the aircraft), configures ± 2000 dps / ± 16 g at 1 kHz ODR in low-noise mode, then samples at 500 Hz with a single 14-byte burst read (temperature + accel + gyro).

Startup gyro-bias calibration

Before the loop, it averages 300 stationary samples (~ 0.6 s) to estimate the gyro's zero-rate bias, which it then subtracts from every reading. A MEMS gyro has a small non-zero output at rest; integrated over time in the attitude filter, that bias becomes drift. Crucially, it sanity-checks the result: if any axis averages above 0.05 rad/s the board was clearly moving during calibration, so it discards the estimate and uses zero bias with a warning rather than baking motion into the bias.

```
const GYRO_SCALE: f32 = (2000.0/32768.0)*(PI/180.0); // LSB → rad/s
const ACCEL_SCALE: f32 = (16.0/32768.0)*9.80665;      // LSB → m/s²
// gyro output = raw*GYRO_SCALE - startup_bias (bias removed so attitude doesn't drift)
```

SAFETY-CRITICAL

On the real flight build, a missing IMU is fatal: the task sets `FlightState::Fault` and parks. On the bench (`nucleo-vcp`) build it warns and continues so motor tests can run without an IMU — with a loud "DO NOT FLY".

5.2 • Barometer — MS5611

The MS5611 is a pressure sensor read over the *shared* SPI1 bus (CS on PA8). It does not output pressure directly: you trigger a D1 (pressure) and D2 (temperature) ADC conversion, wait ~ 10 ms each (OSR=4096), read the 24-bit results, and apply the datasheet's second-order temperature compensation using six factory calibration coefficients read from the chip's PROM at boot. The driver validates the PROM read (all-zeros means a wiring fault) and, being non-essential for basic flight, simply parks itself rather than faulting if the baro is absent.

Altitude is then derived from pressure via the ISA troposphere model:

```
altitude_m = 44330 * (1 - (P / P_ref)^0.190294)
```

where `P_ref` is the ground pressure averaged over the first 30 samples at boot. This makes the reported altitude *height above launch (AGL)*, not mean-sea-level — a datum choice the whole navigation stack depends on (touchdown thresholds, takeoff altitude).

INTERVIEW ANGLE

While a conversion is in flight the task `await s` a timer, which *releases the SPI mutex*, so the IMU can use the bus in the gap. This is a nice illustration of why async cooperative scheduling fits sensor drivers: the 20 ms/cycle the baro spends waiting is not stolen from anyone.

5.3 • Magnetometer — QMC5883L

The compass provides absolute heading so yaw doesn't drift. Two subtleties:

- **Hard/soft-iron calibration every boot.** There is no flash persistence, so on every power-up the operator must rotate the aircraft through a slow figure-8 for a 25 s window while the driver records per-axis min/max. From those it computes a hard-iron offset (the min/max midpoint, subtracted) and a diagonal soft-iron scale (per-axis range normalised, multiplied). If any axis range is too small — meaning the operator didn't actually rotate — it publishes `valid = false`, which keeps GPS-mode arming blocked.
- **Tilt compensation.** A magnetometer only reads true heading when level. The driver takes the live attitude quaternion, extracts roll and pitch, and rotates the magnetic vector back to the horizontal plane before computing heading with `atan2`. Without this, banking the aircraft would swing the heading estimate.

5.4 • GPS — u-blox M10 / UBX

The driver talks the u-blox binary UBX protocol, not NMEA text. At boot it *transmits* two pre-computed `UBX-CFG-VALSET` frames to the module (over the same UART) to enable `NAV-PVT` output and set a 5 Hz navigation rate, then parses incoming `NAV-PVT` messages: a sync-word + class/id/length header, a 92-byte payload, and a Fletcher-8 checksum. It extracts lat/lon (1e-7 deg), MSL height, NED velocity, and horizontal accuracy, and marks the fix usable only when the `gnssFixOK` flag is set *and* the fix type is $\geq 3D$.

GOTCHA — RING-BUFFERED DMA RX

Every serial sensor here uses `into_ring_buffered()`: the DMA continuously fills a background ring buffer so no bytes are lost while the CPU is off doing something else. A naive "read one byte at a time" loop drops bytes between reads under load and corrupts binary frames. This single decision is why the UBX, MICOLINK, SBUS, and MAVLink parsers are reliable.

5.5 • Optical flow & rangefinder — MTF-02P

An optical-flow sensor measures how the ground texture moves across a downward camera (giving body-frame horizontal velocity) plus a laser/ToF range to the ground. It speaks the MICOLINK protocol at 115200; the driver parses message `0x51`, extracting a quality metric, X/Y flow rates, and range in mm, marking data valid only when quality ≥ 50 and range > 0 . It has a "silence detector" that loudly distinguishes a dead wire (no bytes at all) from a wrong-protocol stream (bytes arrive but checksums fail) — a genuinely useful bring-up affordance.

Flow is *not* flight-critical: every consumer guards on `flow.usable()` and falls back to GPS velocity or baro altitude. It buys GPS-denied position hold and precise low-altitude AGL (needed for the weed-descent and precision landing).

5.6 • RC — SBUS

SBUS is Futaba's digital RC protocol: an inverted 100 kbaud 8E2 serial stream of 25-byte frames packing sixteen 11-bit channels. The STM32's UART inverts RX in hardware (`invert_rx = true`), so no external inverter is needed. The driver uses `read_until_idle` — the UART signals when the line goes idle after the 25-byte burst, which naturally frames each ~ 14 ms SBUS packet — with a 100 ms timeout.

The failsafe logic here is layered and worth studying:

- The receiver's own `failsafe` flag acts *immediately*.
- A momentary `frame_lost` flag holds the last channels for a 250 ms grace window before escalating — so a single dropped frame doesn't jolt the aircraft.

- A degraded (partial/error) link trips failsafe after 3 consecutive bad reads (~40 ms).
- A fully silent link (timeout) trips failsafe every tick.

Channels map to throttle/roll/pitch/yaw, an arm switch (ch5), and a 3-position mode switch (ch6). Moving the physical mode switch also *revokes* any MAVLink mode override — the pilot's hand always wins.

5.7 · Battery & current — ADC

A resistive divider scales the pack voltage into the 3.3 V ADC range; the driver reverses the divider ratio ($V_{DIVIDER} = (R1+R2)/R2$) to recover pack voltage, divides by cell count, and maps per-cell voltage to a state-of-charge percentage. Critical-voltage detection uses **asymmetric hysteresis**: 3 consecutive low samples to *enter* critical, 10 consecutive good samples to *leave* it — so a momentary voltage sag under a throttle punch doesn't false-trip, but recovery is deliberately slow and conservative. It is seeded *critical* at boot so there's no window where a 0 V reading looks armable.

INTERVIEW ANGLE

The default `BatteryData` is `{ voltage: 0, critical: true }` — "pessimistic until proven otherwise". Combined with the arming gate, an uncalibrated or unwired divider reads ~0 V, latches critical, and simply refuses to arm. Failing safe by construction, not by a runtime check you might forget, is the pattern to point at.

5.8 · Freshness: the `usable()` discipline

This is the most transferable idea in the sensor layer. Because every task leaves its *last* snapshot in `STATE`, a task that dies leaves stale-but-plausible data behind forever. A consumer that trusts a bare `valid` flag would fly on a frozen GPS fix. So every snapshot carries a `stamp_ms` timestamp, and consumers never read the flag directly — they call a helper:

```
pub fn usable(&self) -> bool {           // GpsFix
    self.fix_ok && is_fresh(self.stamp_ms, GPS_FRESH_MS) // 2500 ms
}
// analogous: FlowData::usable(), MagData::usable(), ImuData::is_fresh()
```

"Valid *and* recent" is the invariant. A frozen sensor becomes `!usable()` within a bounded window and every downstream mode falls back gracefully.

6 State estimation

6.1 · Reference frames & the NED/NWU trap

Three frames are in play, and mixing them is the single most common source of sign bugs in autopilots:

Frame	Axes	Used by
Body	X forward, Y right, Z down (aircraft-fixed)	gyro/accel, motor mixing
NWU (Madgwick native)	X North, Y West, Z Up	the AHRS filter internally
NED (navigation)	X North, Y East, Z Down	GPS, estimator, MAVLink, guidance

GOTCHA — ONE HELPER TO RULE THEM ALL

The Madgwick filter produces a yaw in its native *CCW-positive NWU* convention, but all navigation math is *CW-positive NED* (0 = North, clockwise). The firmware forces every navigation consumer through a single `ahrs::ned_yaw(q)` helper that negates and reframes the yaw, with a bring-up note: "rotate the airframe clockwise and confirm this *increases*." Likewise the estimator converts specific force from NWU to NED with `N=X, E=-Y, D=-Z`. Both conventions were physically bench-verified (lift the board → down-velocity goes negative). Being able to explain *why* a frame mismatch produces a slow yaw-off or a runaway climb is a strong signal.

6.2 · Madgwick AHRS

Attitude comes from a Madgwick filter — a computationally cheap alternative to an attitude EKF. The intuition: the gyro gives an excellent *short-term* rate of rotation but drifts over time; the accelerometer gives a noisy but *drift-free* gravity reference; the magnetometer gives a drift-free heading reference. Madgwick fuses them by (1) integrating the gyro to propagate the quaternion, and (2) applying a single gradient-descent step that nudges the quaternion to better agree with the measured gravity (and magnetic) direction. A gain `β` trades convergence speed against noise.

```
q̇ = ½ q ⊗ ω // gyro propagation
∇f = JT · f(q, gravity[, mag]) // gradient of the alignment error
q += (q̇ - β · ∇f / |∇f|) · dt ; normalise // fused update, run at 500 Hz
```

The implementation has both a 6-DOF (`update`) and a 9-DOF (`update_with_mag`) path; `control_task` uses the mag path only when the compass is `usable()`, and the mag path itself falls back to 6-DOF on a degenerate magnetic vector. The rate loop feeds on a low-pass-filtered gyro while Madgwick uses the raw gyro — deliberately, so the filter isn't given extra phase lag.

INTERVIEW ANGLE — WHY NOT AN EKF?

A good answer: "An EKF gives you a principled covariance and better handling of correlated noise, but it's heavier, needs tuning of process/measurement noise, and is easy to get subtly wrong. Madgwick is $O(n)$, deterministic, and more than adequate for a quad's attitude at 500 Hz. I kept the position estimator complementary rather than an EKF for the same reason — I traded optimality for something I can fully reason about and that can't diverge silently."

6.3 · Complementary position/velocity estimator

A separate 150 Hz task fuses IMU, GPS, and flow into an NED position/velocity estimate, deliberately *not* an EKF. It is a predictor–corrector:

- **Predict (150 Hz):** rotate body specific force into the world frame with the attitude quaternion, subtract gravity on the Down axis, and integrate acceleration → velocity → position (forward Euler). Critically it only predicts when the IMU snapshot `is_fresh()` — integrating an all-zero or frozen accel sample would manufacture phantom velocity.
- **Correct from GPS (5 Hz):** blend the dead-reckoned estimate a fixed fraction toward the GPS position and velocity (`K_POS=0.10` , `K_VEL=0.30`). The first valid fix sets the NED origin.
- **Correct from flow (low altitude):** when GPS corrections aren't recent, fuse flow-derived horizontal velocity weakly.
- **Dropout handling:** during a long GPS outage the velocity is bled toward zero ($\times 0.999/\text{tick}$) so a biased accelerometer can't run the position estimate away unbounded.

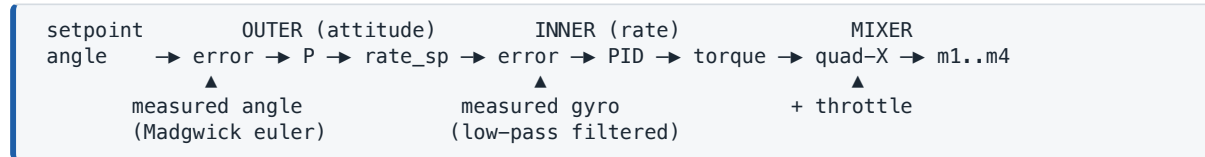
INTERVIEW ANGLE

The complementary-gain framing is clean: "each correction gain sets a first-order time constant — position tracks GPS with roughly a 2-second time constant, velocity with about 0.7 s. High-frequency motion comes from the IMU, low-frequency truth from GPS, and the crossover is those gains." That's the whole philosophy of a complementary filter in one sentence.

7 Flight control

7.1 • The cascade: angle → rate → torque

The controller is a classic two-loop cascade running inside `control_task` at 500 Hz:



Outer loop turns an angle error into a desired body rate; inner loop turns a rate error into a torque demand.

Yaw is handled slightly differently — the pilot/nav commands a yaw *rate* directly, bypassing the outer angle loop (you don't hold an absolute heading with the sticks, you command a turn rate).

7.2 • PID implementation details

The `Pid` struct is textbook in structure but has three production-grade refinements that are exactly what interviewers dig for:

- **Derivative on measurement, not error.** D acts on $-d(\text{measurement})/dt$, so a step change in the setpoint doesn't produce a derivative "kick" (a huge transient spike). Standard practice, but a mark of someone who's tuned real loops.
- **Derivative-on-change for slow sensors.** The D term is recomputed only when the measurement actually changes and *held* in between, dividing by the true elapsed span. Naively differencing every tick against a 5 Hz GPS or 25 Hz baro would produce a sawtooth of zeros and spikes. This lets one `Pid` type serve both the 500 Hz rate loop and the slow altitude/position loops.
- **Anti-windup by clamped integral.** The integral is clamped to `i_limit`, chosen so $K_i \cdot i_limit$ is a bounded fraction of the output authority — the I-term alone can never saturate the actuator.

```
// Inner rate loop gains (roll/pitch): Kp=50, Ki=30, Kd=2, i_limit=16, out=±500
// Outer attitude loop: P-only, Kp=4 (Ki/Kd = 0 – a P shell is enough)
// Altitude PID wraps a hover feed-forward: throttle = 0.55 + pid(target,alt), clamped [0.1,0.9]
```

There's also a ground/idle anti-windup guard: below 5% throttle the PIDs are reset, because on the ground the rate integrals wind to their clamps and pin a motor at full scale (this was observed props-off and fixed). And a **crash cutoff**: if roll or pitch exceeds $\sim 75^\circ$ for 0.2 s while armed, motors are killed and the aircraft latches Fault.

7.3 • Quad-X mixing & desaturation

The mixer converts (throttle, roll, pitch, yaw) torque demands into four motor outputs by the quad-X geometry — each motor contributes to throttle plus a signed combination of the three torques based on its position and spin direction:

```
r1 = thr - roll + pitch - yaw  (M1 front-right, CW)
r2 = thr + roll + pitch + yaw  (M2 front-left,  CCW)
r3 = thr - roll - pitch + yaw  (M3 back-right,  CCW)
r4 = thr + roll - pitch - yaw  (M4 back-left,   CW)
```

If any motor would exceed $[0,1]$, naively clamping each independently *distorts the torque ratios* and the aircraft rolls/pitches wrong at high throttle. Instead the mixer computes a single common offset and shifts *all four* outputs by it before clamping — preserving the differential (attitude authority) at the expense of a little collective (altitude). Prioritising attitude over throttle when saturated is the correct choice: you'd rather sag in height than lose control authority. This is exactly the kind of detail that shows you understand why the obvious implementation is wrong.

8 Navigation & flight modes

8.1 · Mode set and the demotion ladder

`navigation_task` (100 Hz) is the autonomy brain. It writes an `AttitudeSetpoint` into `nav_command` that `control_task` consumes when `autonomous` is set; otherwise the pilot's sticks pass through.

Mode	Behaviour	Needs
Stabilise	Self-levelling; manual throttle & yaw	IMU only
AltitudeHold	Baro altitude lock; RC roll/pitch/yaw	+ baro
PositionHold	GPS+flow position lock; RC yaw	+ GPS, mag
Auto	Fly uploaded waypoint mission; weed targets take priority	+ GPS, mag
ReturnToHome	Climb ≥ 15 m, fly home on haversine bearing, descend	+ GPS, mag
Land	Rate-controlled descent, rangefinder touchdown detect	+ baro
FollowMe	Track GCS-supplied person position (YOLOv8)	+ GPS, mag

SAFETY-CRITICAL — GRACEFUL DEMOTION

Modes degrade rather than fail. Lose GPS for >5 s in a GPS mode $\rightarrow$ auto-demote to AltitudeHold. Lose the altitude source mid-flight $\rightarrow$ hand back manual throttle if a pilot is on the sticks, or force Land if there's no RC link. Enter Fault $\rightarrow$ force Land and auto-disarm at touchdown. The aircraft always falls back to the most capable mode its *currently healthy* sensors can support.

8.2 · Guidance law

The core `guide_to(target)` routine, shared by waypoints, RTH, and weed phases: convert the geodetic error to local North/East metres (flat-earth approximation, valid at short range), clamp it to a cap so a distant target doesn't demand an absurd lean, rotate into body frame using the NED yaw, run the horizontal position PIDs to produce roll/pitch lean angles (clamped to a max tilt), run the altitude PID for throttle, and set a yaw rate that turns the nose toward the bearing when more than 3 m away.

```
fwd_m   = err_n*cos(yaw) + err_e*sin(yaw)  // rotate NED error into body
right_m  = -err_n*sin(yaw) + err_e*cos(yaw)
pitch    = -pos_pid_n(fwd_m)  // lean forward to fly forward
roll     = pos_pid_e(right_m)
throttle = alt_pid(target_alt, current_alt)
```

8.3 · The weed-extraction state machine

The headline autonomous capability. When a weed target arrives (a GPS coordinate from the ground station's vision) while in Auto mode, the aircraft runs an 8-phase sequence, latching the target so a new detection mid-sequence can't re-aim a live descent:

```

Approach → Descend → Stabilize → Extract —(grip ok)→ Ascend → Dispose →
DisposeDescend → done
(cruise (to (hold 1s (servo | (fly to (descend over
to GPS extract settle) close) | home/bin) bin,
release) lat/lon) alt AGL) └(grip fail)→ GripRelease → (retry ≤3) →
or abort empty

```

The gripper state machine, optionally closed-loop via a jaw microswitch under `--features grip-sense`.

With `grip-sense`, a microswitch on the jaw confirms each grab; a failed grab reopens, dwells, and retries up to 3 times, then aborts that weed and climbs away empty (skipping the bin run) rather than flying a phantom payload home. Without the feature it trusts the timer (open-loop). The descent uses rangefinder AGL when available for the tight ± 0.15 m altitude band, falling back to baro.

INTERVIEW ANGLE

This is a good "tell me about a complex feature" story: an explicit finite-state machine with latched inputs, per-phase timers, a bounded retry policy, and graceful failure (abort empty, don't strand the mission). Contrast it with what you'd get from ad-hoc `if` -ladders — the FSM makes every transition and failure mode auditable.

8.4 · Geofence, failsafes, and latching

Independent of the selected mode, the task continuously checks:

- **Geofence:** altitude > 120 m or horizontal distance from home > 300 m → force RTH (or Land if no GPS).
- **Battery:** <30% SoC → RTH; critical voltage → immediate forced RTH/Land.
- **Flight timer** (no-batt builds): armed longer than a configured limit → RTH/Land, standing in for the missing voltage sensing.

GOTCHA — WHY FAILSAFES LATCH

Each failsafe response is *latched* until disarm. A recovering geofence breach or a battery percentage that bounces back up under reduced load would otherwise cause the mode to oscillate at the trigger boundary — RTH, recover, resume, breach, RTH... Latching plus "a pilot-selected Land still overrides a latched RTH (strictly more conservative)" gives a stable, predictable failsafe hierarchy. And a failsafe-forced landing disarms *and* Fault-latches at touchdown, so the aircraft can't be re-armed without a power cycle (which, since the BEC feeds the FC, means a physical pack swap).

9 Actuation: the DShot pipeline

This is the most hardware-intimate part of the firmware and a superb interview topic because it spans protocol, timer hardware, DMA, and a real debugging saga.

9.1 · DShot on the wire

DShot is a digital ESC protocol — instead of a 1000–2000 μs analogue PWM pulse, it sends a 16-bit frame: 11 bits of throttle (48–2047), 1 telemetry-request bit, and a 4-bit CRC. Each bit is a fixed-period pulse whose *duty cycle* encodes the value — a long high (75%) is a 1, a short high (37.5%) is a 0. "DShot300" means 300 kbit/s. Because it's digital + checksummed, it's immune to the analogue noise and calibration drift of PWM.

```
frame = (throttle<<1 | telem_bit); crc = (f ^ f>>4 ^ f>>8) & 0xF; frame = frame<<4 | crc
```

9.2 · Timer burst-DMA encoding

Bit-banging four 300 kbit/s streams in software would burn the CPU. Instead the firmware uses **TIM3 in PWM mode with burst DMA**. The trick: each DShot bit becomes one timer period, and the bit's value is the timer's compare register (CCR) value for that period — T1H ticks for a 1, T0H for a 0. A DMA buffer holds the CCR values for all 16 bits $\times$ 4 channels, and on every timer update event the timer's burst-DMA feature (DBA/DBL registers) fans one DMA transfer out across CCR1–CCR4, driving all four motors in lockstep from a single stream.

```
ARR = 332  (333 ticks @ 100 MHz PSC=1 → 3.33  $\mu\text{s/bit}$  = DShot300)
T1H = 250  (75% duty = logic 1)   T0H = 125  (37.5% = logic 0)
Buffer: [bit0.m1, bit0.m2, bit0.m3, bit0.m4, bit1.m1, ...] – 18 slots  $\times$  4 = 72 u16
DShot600 = same duty ratios, PSC=0 (they're speed-independent) – one const changes
```

9.3 · The H7 DMA gotchas

Getting this working on the H7 involved several real bugs, all documented in the code and each a good war story:

- **Wrong DMAMUX request ID.** The DMAMUX must route stream 4 to TIM3_UP (request 27, per RM0468 Table 105). It was initially 22; the DMA never fired and the motors were silently dead. *Lesson: on H7 you verify the DMAMUX request line against the reference manual, not by guessing.*
- **32-bit writes into a 16-bit register.** The timer's DMAR register is 16-bit; the DMA must do halfword (16-bit) transfers. 32-bit transfers corrupt the burst fan-out.
- **DTCM buffer invisible to DMA** (§3.1) — the buffer must live in AXI-SRAM.
- **WFE clock-gating.** When the executor sleeps (WFE) between frames it clock-gates TIM3/DMA and can strand a frame mid-transfer; the task deliberately spins until the frame drains before yielding.
- **ESC only parses DShot300.** The fitted BLHeli_S ESC advertises "DShot600/300" but its target MCU only decodes 300 — so 300 is the default and 600 is an opt-in feature for a future ESC.

INTERVIEW ANGLE

The debugging methodology is the story: because the failure was silent, the firmware grew a `dshot-debug` feature that dumps the GPIO alt-function/timer/DMA/DMAMUX config at init, runs a static-PWM bisection (drive a 50% duty with no DMA and probe the pins — toggle means the output stage is fine and DMA is the fault), samples the GPIO input register to confirm pads physically toggle, and surfaces 1 Hz DMA health counters over MAVLink

STATUSTEXT so the bench is diagnosable with no debugger attached. "Make the invisible visible, then bisect" is the whole lesson.

The firmware also never busy-waits to send: `dshot_send()` returns "skipped" if the previous DMA is still running, so a stuck DMA can't wedge the single-threaded executor or starve the watchdog pet. And a props-off ESC arming sequence streams 2 s of continuous zero-throttle frames at boot, because BLHeli_S treats a silent line as signal-loss and won't arm.

9.4 • Bidirectional DShot / eRPM

There is a scaffold for bidirectional DShot (inverted CRC so the ESC replies with GCR-encoded eRPM telemetry). The encode and GCR/eRPM decode math are implemented and unit-correct; the on-wire edge capture is explicitly marked as not-yet-bench-validated and returns `None` so it can never masquerade as working telemetry. Honest scaffolding — labelled as such — is worth more than fake completeness.

10 Telemetry: MAVLink v2 by hand

MAVLink is the de-facto drone telemetry protocol. Rather than pull in a code-generating crate, the firmware hand-implements the v2 frame format — which is both a size win on a `no_std` target and a demonstration of understanding the protocol byte-for-byte.

10.1 • Frame format & CRC

```
0xFD | len | incompat | compat | seq | sysid | compid | msgid[3] | payload | crc[2] | [sig]
CRC = CRC-16/MCRF4XX over bytes[1..end-of-payload] with a per-message CRC_EXTRA "magic" byte
appended
```

The `CRC_EXTRA` magic is a hash of the message's field definition baked into both ends; it means a sender and receiver with mismatched message *definitions* fail CRC rather than silently misparsing. The firmware hard-codes the magic for each message it speaks (`HEARTBEAT=50`, `ATTITUDE=39`, ...).

10.2 • The staggered TX schedule

One 10 Hz ticker drives everything, but messages are spread across ticks so the UART never has to send a 7-message burst in one tick (which would blow the 57600-baud budget):

Message	Rate	Carries
ATTITUDE #30	10 Hz (every tick)	roll/pitch/yaw + rates
VFR_HUD, SERVO_OUTPUT_RAW, GLOBAL_POSITION_INT	5 Hz (even ticks)	alt, speed, heading, motor/servo μ s, position
HEARTBEAT, SYS_STATUS, GPS_RAW_INT, BATTERY_STATUS, SCALED_IMU	1 Hz (staggered)	mode/state, sensor-health bitmask, fix, pack, raw IMU/mag

The HEARTBEAT packs the firmware's own state into the autopilot-defined `custom_mode` : flight mode in bits [7:0], flight state in [15:8], payload-presence flags in [31:16] — so the GCS reconstructs the exact internal state from a standard message.

10.3 • The RX state machine & command dispatch

RX is a byte-by-byte state machine (`SYNC` → `HEADER` → `PAYLOAD` → `CRC` → optional `SIGNATURE`) fed from a ring-buffered DMA RX. It verifies CRC (rate-limiting the warning on a corrupt stream so a baud mismatch can't flood the log), zero-pads truncated v2 payloads so fixed-offset reads don't see stale bytes, and dispatches by message id. TX and RX run as two `async` futures `joined` inside the one task.

Commands handled via `COMMAND_LONG` : arm/disarm (400), set servo (183), motor test (209), RTL (20), land (21), set mode (176), set home (179). Every one gets a `COMMAND_ACK` ; unknown commands get `UNSUPPORTED` — no silent drops.

SAFETY-CRITICAL — COMMAND VALIDATION

The command handler is not a passive executor. Arm re-runs the full pre-arm check *and* requires the RC arm switch on. In-air disarm is *rejected* unless the GCS supplies a "force magic" value (21196) — so a stray or replayed `COMMAND_LONG` can't cut motors in flight. A motor-test command is refused unless disarmed and Idle. The FC treats every external command as a request to be validated, never an order.

10.4 • Mission upload protocol

The firmware implements the MAVLink mission microprotocol as the receiver: `MISSION_COUNT` starts a handshake, the FC requests each item with `MISSION_REQUEST_INT`, stores `NAV_WAYPOINT` items contiguously (ignoring seq gaps from non-nav items), and finalises with `MISSION_ACK`, setting a `ready` flag that `navigation_task` picks up. The weed and follow targets ride in on `SET_POSITION_TARGET_LOCAL_NED`, whose NED offsets are range-checked (≤ 50 m, finite) before being converted to a target lat/lon against the current fix.

11 Safety architecture (read this twice)

If there is one thing to be able to talk about fluently, it is this. The safety model is *layered* and *defensive*: multiple independent mechanisms, each of which can stop the aircraft, arranged so no single failure defeats them all.

11.1 • The layered kill-path model

Layer	Mechanism	Trips on
Pilot	RC arm switch (absolute, honoured in every state incl. Fault)	Any time the pilot flips it
RC link	SBUS failsafe (immediate / graded / timeout)	Radio loss or degradation
Attitude	Crash cutoff in control_task	>75° tilt for 0.2 s while armed
Power	Battery critical / low	Per-cell V or SoC thresholds
Geometry	Geofence	Altitude >120 m or >300 m from home
Companion	Pi heartbeat watchdog	No Pi heartbeat for 5 s → Fault→Land
Software liveness	IWDG hardware watchdog	control_task hangs >2 s → chip reset
Last resort	Panic handler	Any Rust panic → emergency motor stop

11.2 • Arming gates

Every arm path — RC and MAVLink — funnels through one `pre_arm_check(mode)` so the two can never diverge. It requires: RBF pin removed (when enabled), IMU healthy, a live barometer for any non-Stabilise mode, nav-ready GPS + calibrated compass for GPS modes, and a non-critical battery. Arm is also gated on throttle <5% and no failsafe. The check is *mode-aware*: Stabilise needs only the IMU, so you can always bench-hover, but PositionHold refuses without GPS and a figure-8'd compass.

SAFETY-CRITICAL — CURRENT BENCH POSTURE

Right now `SAFETY_PIN_ENABLED = false` (RBF gate bypassed for bench work) and the build is often `rc-optional` (RC gates waived until an SBUS link is seen this boot, making the GCS disarm and Pi-loss watchdog the only kill paths). Both are deliberate, loudly-warned bring-up states. Re-enabling the RBF gate and building without `rc-optional` are on the pre-field-flight checklist. Knowing precisely which safety layers a given build has stood down — and why — is the mark of taking safety seriously rather than performatively.

11.3 • In-flight failsafes

Covered in §8.4 (latched, hierarchical, disarm-and-Fault at touchdown). The key property: failsafes prefer the *survivable* action given available sensors — RTH if GPS+home exist, else Land; hand back manual throttle if a pilot is present, else Land. And Fault recovery is *source-owned*: a Pi-loss Fault auto-clears when the link returns (and only on the ground, disarmed), but an IMU or battery Fault stays latched and needs operator intervention. You never silently recover from a hardware fault.

11.4 • The hardware watchdog

The STM32's independent watchdog (IWDG) runs off its own low-speed oscillator, started *before* Embassy init, with a 2 s timeout. Only `control_task` pets it (every 0.5 s). The reasoning is deliberate: `control_task` is the one task that *must* keep running for the aircraft to be controllable, so tying the watchdog to it means any hang

that stops control — even one that leaves other tasks alive — triggers a hardware reset. The panic handler is the final net: any Rust panic fires an explicit zero-throttle DShot frame (draining any in-flight DMA first) and turns the LED off, via raw register writes because at panic time you can't rely on the HAL.

12 Companion computer & ground station

12.1 • The Pi relay

The Raspberry Pi runs a small Python relay (`weed_pilot.py`) with four daemon threads sharing one serial port under a lock:

- **serial→UDP:** forward STM32 telemetry bytes to the laptop (UDP 14550).
- **heartbeat:** send a MAVLink HEARTBEAT to the STM32 at 4 Hz — many retries inside the 5 s watchdog window, so a single dropped heartbeat never trips a false Pi-loss Fault.
- **UDP→serial:** forward GCS commands (arm, mode, mission) to the STM32, with a source-IP allowlist.
- **weed-target listener:** receive weed coordinates from the GCS (JSON/UDP) and translate them into `SET_POSITION_TARGET_LOCAL_NED` MAVLink frames for the FC.

It also self-monitors: if any thread dies it exits so systemd restarts it cleanly. A separate service streams the camera as hardware-encoded H.264. The Pi README is a study in field-hardening: WiFi power-save disabled via a boot service (parked radios cause random drops), the stable PL011 UART freed from Bluetooth (the mini-UART's baud drifts with CPU clock and corrupts MAVLink), camera bitrate capped so video doesn't starve the shell of WiFi airtime, and a Pi 4 with wired Ethernet adopted for the bench precisely to separate the shell from the video radio.

12.2 • The Python GCS & computer-vision pipeline

The laptop ground station (DearPyGUI) is the operator interface and the vision engine:

- **Telemetry HUD & pre-arm overlay:** parses the MAVLink stream (pymavlink) into a live dashboard and a 7-item readiness checklist (LINK, GPS, IMU, BARO, BATTERY, THROTTLE, HOME) that blocks arming until green — mirroring the firmware's own gates so the lamps agree with what the FC will actually accept.
- **Mission builder:** uploads waypoint missions via the MAVLink mission protocol; a program panel issues RTL/Land/mode commands.
- **Weed detection (ExG):** the "Excess Green" index ($2G - R - B$) segments vegetation in each camera frame; blobs are click-assignable persistent IDs, tracked across frames by optical flow + template re-identification, and georeferenced to lat/lon using GPS + attitude to build a top-down weed map exported as GeoJSON.
- **Person tracking (YOLOv8):** for FollowMe, a YOLOv8 model detects a person; a selected track's world position is streamed to the FC as a follow target.

GOTCHA — METAL THREADING ON MACOS

YOLO inference runs in an isolated *subprocess* communicating over pipes, because PyTorch and DearPyGUI sharing a Metal context across threads on Apple Silicon SIGSEGVs. BLAS thread counts are pinned to 1 for the same reason. A real-world reminder that "just call the model inline" doesn't survive contact with GPU-context threading rules.

13 Build system & feature flags

Cargo *features* are used as compile-time configuration, which on an MCU means zero runtime cost and no dead code shipped. Knowing these demonstrates the bench-vs-flight discipline:

Feature	Effect	Class
(default)	DShot300, flight build, all safety gates active	flight
dshot600	Raise ESC bit rate (one prescaler change)	hardware variant
nucleo / nucleo-vcp	Remap SPI MOSI (PA7→PD7, Ethernet-PHY conflict) & LED; route MAVLink to ST-Link USB VCP for bench	board variant
grip-sense	Closed-loop gripper via PC2 microswitch + retry	capability
range-alt	Use rangefinder as altitude source when baro is absent; clamp all altitudes to a 3 m ceiling	degraded-sensor
rc-optional	Waive RC gates until an SBUS link is seen (radio lost)	degraded-sensor
no-batt	Disable voltage sensing; flight-timer becomes the only pack protection	degraded-sensor
tune-log / dshot-debug / dshot-bidir	PID blackbox stream / DShot TX instrumentation / eRPM scaffold	diagnostic
bench-force / spin-all / pin-test	Bench-only motor drive modes (props off, never flight)	bench (dangerous)

[profile.release] uses `opt-level='s'` (size), LTO, and keeps debug symbols (for defmt). And a hard-won note lives in the dev profile: `opt-level > 0` there breaks embassy-stm32 init on this chip, so it's never added.

14 Known issues & engineering judgement

Being able to talk about what *isn't* done, and why, is often the most credible part of an interview.

- **Executor cadence "clumping" (open).** Occasionally the 500 Hz loop's inter-frame gaps become bursty rather than a steady 2 ms. It's instrumented (max-gap / long-gap / short-gap counters surfaced over MAVLink, precisely so the measurement doesn't perturb the thing being measured the way an RTT probe would). The two live suspects are *defmt-RTT blocking under the debug probe* and a *UART error-interrupt storm*; the planned discriminator is to compile logging out (`DEFMT_LOG=off`) — if it goes steady, RTT was the culprit. Not yet root-caused.
- **Corner-mapping re-verification.** Motor direction is fixed, but a new ESC harness means the `MOTOR_SLOT=[1,3,0,2]` remap (which STM32 output drives which physical corner) needs a fresh `motor_test.py --sequence` before props go on. Motor *direction* $\neq$ corner *order*.
- **Calibrations pending:** `V_DIVIDER` against a meter, `CUR_A_PER_V` before trusting current, compass figure-8 each boot.
- **Deliberate non-choices:** complementary filters instead of an EKF; hand-rolled MAVLink instead of a generator; polling the IMU at 500 Hz instead of using its data-ready interrupt. Each is a defensible trade of optimality/generality for simplicity and reason-ability on a resource-bounded target.

15 Interview preparation: likely questions & strong answers

"Walk me through what happens from power-on to hovering."

Power-on → clocks configured (400 MHz PLL, supply config, watchdog started first) → SPI bus + GPIOs init → all tasks spawned → IMU verifies WHO_AM_I and runs a 0.6 s gyro-bias cal (keep still) → compass figure-8 cal (25 s) → sensors stream into `STATE`, `health_task` gates arming → operator arms via RC or GCS through the shared pre-arm check → motors idle at 4% → throttle past 15% → state goes Flying → `control_task` fuses attitude (Madgwick) and runs the angle→rate PID cascade at 500 Hz → mixer → DShot frames out via TIM3 burst DMA → aircraft holds attitude; select AltitudeHold/PositionHold to close the baro/GPS outer loops.

"How do you keep a 500 Hz control loop deterministic on a single core?"

Cooperative async executor with no heap and no per-task stacks; the control loop is timer-paced with an absolute-deadline Ticker; sensor drivers `await` during their long waits (releasing shared buses) so they don't steal loop time; the loop never busy-waits on DMA (it skips a frame instead); and an independent hardware watchdog tied specifically to the control task catches any hang. The one honest caveat is the open cadence-clumping investigation.

"Why Rust and not C?"

See §4.1 — memory/concurrency safety at compile time, no GC pauses, explicit fallibility, auditable `unsafe`. Acknowledge the costs: ecosystem maturity, learning curve, and needing register-level knowledge for peripherals the HAL doesn't cover (which is exactly what the DShot work required).

"Tell me about a hard bug."

The DShot silence (§9.3): motors dead with no error. Wrong DMAMUX request line, 32-bit writes into a 16-bit register, and the DTCM-invisible-to-DMA trap, found by building an instrumentation feature that dumps peripheral config, bisects with a static-PWM-no-DMA test, and surfaces DMA health counters over telemetry so it's debuggable without a probe attached. Methodology: make the invisible observable, then bisect.

"How does the system fail safe?"

§11 — layered independent kill paths (pilot switch, RC failsafe, crash cutoff, battery, geofence, Pi watchdog, IWDG, panic handler), one shared mode-aware pre-arm check, latched hierarchical failsafes that prefer the survivable action, source-owned Fault recovery, and fail-safe defaults (battery seeded critical, sensors `usable()` -gated on freshness). Be candid about which layers a bench build stands down and why.

"Frames and conventions?"

Body (FRD), Madgwick's native NWU, and navigation NED — funnelled through a single `ned_yaw()` helper and an explicit NWU→NED conversion in the estimator, both physically bench-verified. This is where sign bugs live; centralising the conversion is the defence.

"What would you do next / with more resources?"

Close out the cadence investigation; move the IMU to interrupt-driven data-ready; add flash-persistent compass calibration; validate bidirectional-DShot eRPM for closed-loop RPM; and, given a safety case, consider a formal EKF for wind/velocity estimation. Re-enable every bench-disabled safety layer before field flight.

16 Glossary

AHRS	Attitude & Heading Reference System — the attitude estimate from IMU (+mag).
Madgwick	A low-cost gradient-descent complementary filter fusing gyro/accel/mag into a quaternion.
NED / NWU / FRD	North-East-Down / North-West-Up / Forward-Right-Down reference frames.
DSHOT	Digital ESC protocol; 16-bit checksummed frames, duty-cycle-encoded bits.
ESC	Electronic Speed Controller — drives a brushless motor.
SBUS	Inverted digital RC protocol, 16×11-bit channels at 100 kbaud 8E2.
MAVLink	Lightweight drone telemetry/command protocol; v2 framing with per-message CRC_EXTRA.
UBX / NAV-PVT	u-blox binary GPS protocol / its position-velocity-time solution message.
Embassy	Async/await executor + HAL for embedded Rust; cooperative, no-heap.
IWDG	Independent watchdog — resets the MCU if not periodically petted.
DMA / DMAMUX	Direct Memory Access controller / the mux routing peripheral requests to DMA streams.
ExG	Excess Green index (2G−R−B) — a cheap vegetation segmentation heuristic.
RBF	Remove-before-flight pin — a physical arming interlock.
RTH / SoC / AGL	Return-to-home / State of charge / Above-ground level.

Generated from the firmware source on branch **main**. Where documentation and code disagreed, the code was treated as ground truth (e.g. DSHOT300 default, 115200-baud GPS). Every mechanism described corresponds to real code in **src/**, **pi/**, and **ground/**.